Homework 1 : Catching a Moving Target

Gautam Balaji

September 27, 2026

1 Planner Summary

1.1 Theory

Problem. Each step, the robot moves to one of its 8 neighbours or waits, and pays the cost c of the cell it occupies. With the target's trajectory known, the planner minimises

$$J = \sum_{t=t_0}^{t_c-1} c(p_t), \tag{1}$$

where p_t is the robot's cell at time t and t_c is the catch time. Searching over (x, y, t) directly means about 10^{10} states on map 2, so the planner exploits one property of the problem.

Wait at the cheapest cell. Moving all of a plan's waiting to the cheapest cell w on its path keeps the catch time the same without raising the cost. An optimal plan therefore goes to w, waits, and then meets the target. With $m = c(w)$,

$$J = \sum_{\text{path}} (c - m) + (t_c - t_0)\, m, \tag{2}$$

a standard shortest-path cost with non-negative edge weights $c - m$, plus a terminal cost that depends only on the trajectory point caught. The time dimension disappears.

Algorithm. The plan is computed once, on the first call:

1. **Reachability:** a BFS gives the fewest moves to every cell. A trajectory point is a goal only if the robot can reach it before the target does. The cheapest reachable cost is used as m.
2. **Wait-cell decomposition:** a *backward multi-goal Dijkstra* [1], seeded at all goals with $(t_c - t_0)\, m$, gives every cell its cost-to-catch B and a *deadline*; labels whose deadline the robot cannot meet are pruned. A *forward A** with heuristic B then finds the best wait cell for Eq. (2), stopping when no open node can beat the best plan (branch and bound [4]).
3. **Exact space-time search:** keeps one (cost, deadline) label per cell, so under tight timing it can miss the optimum [3]. When the (x, y, t) region is small, Eq. (1) is solved exactly layer by layer in time [2]. States are pruned when their cost plus
$$h(v, t) = D(v) + \big(E(v, t) - t\big)\, c_{\min} \tag{3}$$
reaches the next cost. Here D is the minimum extra cost to the trajectory, and E is the earliest possible catch time from the Chebyshev distance (found by binary search, since the target moves at most one cell per step). h is admissible, so the result is optimal.
4. **Execution:** the plan is a spatial path plus a catch time. The robot waits if it has spare time and is on the cheapest remaining cell, and moves on otherwise.

Every plan is feasible, and optimal whenever the next planner runs.

1.2 Efficiency features

- **Plan once:** all planning happens on the first call, and later calls take $O(1)$.
- **No time dimension:** waiting at the cheapest cell turns the (x, y, t) problem into 2-D searches in the common case.
- **Multi-goal search:** all trajectory points are handled by a single backward Dijkstra instead of one search per goal.
- **Near-exact heuristic:** the forward A* uses the backward costs as its heuristic, so it expands only a small corridor of cells.
- **Time pruning:** goals that cannot be reached in time and labels whose deadline cannot be met are discarded early.

1.3 Memory Management

- All per-cell data lives in `std::vector`s, which release their memory automatically when they go out of scope. No manual `new`/`delete` is used anywhere.
- The planners are local objects inside the first call's planning block, so their working arrays are freed as soon as planning finishes. Only the `GridMap` (which holds pointers to the map, without copying it) and the plan persist, as `static` variables.
- The exact search stores only two dense layers plus a compact 8-byte record per visited state. States are pruned by branch and bound, so only about 1.2 M of the estimated 3.2×10^7 states are stored on map 9.
- **Measured peak memory** of the whole process, including the map loaded by `runtest`: about 150 MB on maps 1 and 2 (1825×2332), 69 MB on map 4, 29 MB on map 9 and 10 MB on map 5.

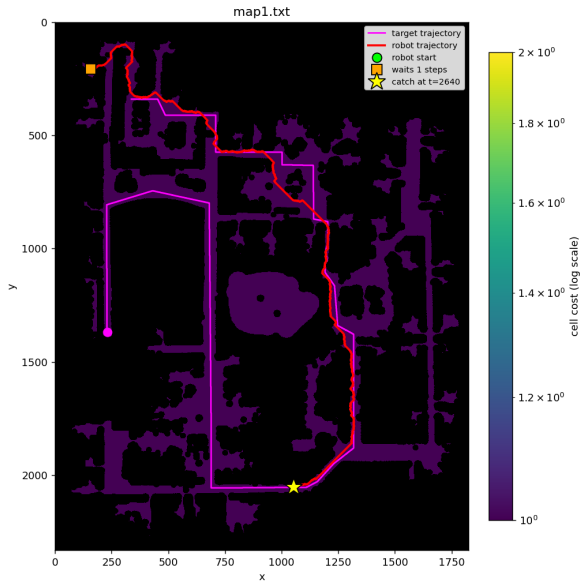
2 Results

The results for all maps are summarised in Table 1.

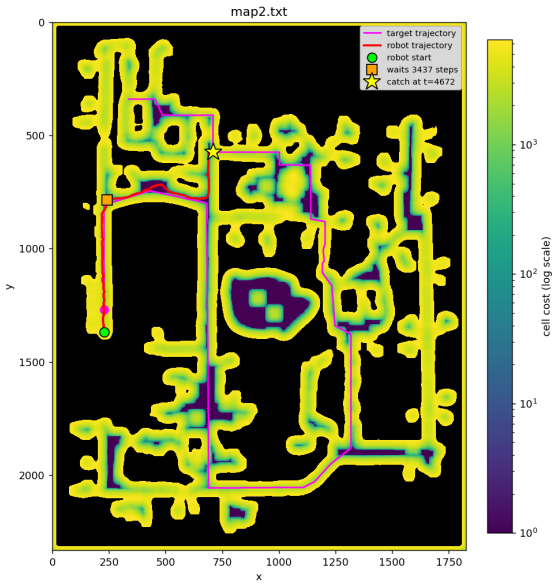
Map	Target caught	Time taken (s)	Moves made	Path cost	Planning time (s)
1	yes	2640	2639	2640	0.110
2	yes	4672	1235	1599074	0.239
3	yes	241	241	241	0.023
4	yes	375	266	375	0.355
5	yes	181	181	1063	0.028
6	yes	140	22	539	0.032
7	yes	250	250	250	0.023
8	yes	431	430	431	0.141
9	yes	368	367	368	0.119

Table 1: Results for all maps (Release build).

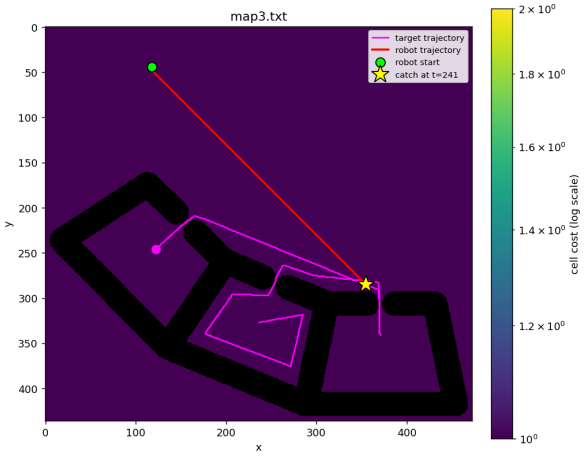
The plots shown in Fig. (1) to Fig. (3) depict each map with the target trajectory (magenta), the robot trajectory (red), the robot start, the waiting location, and the catch point.



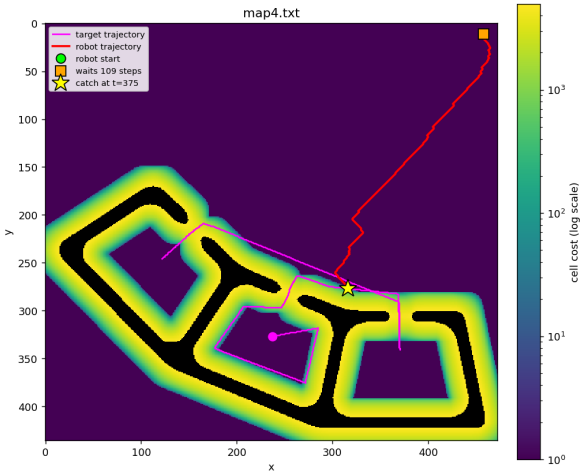
(a) Map 1



(b) Map 2

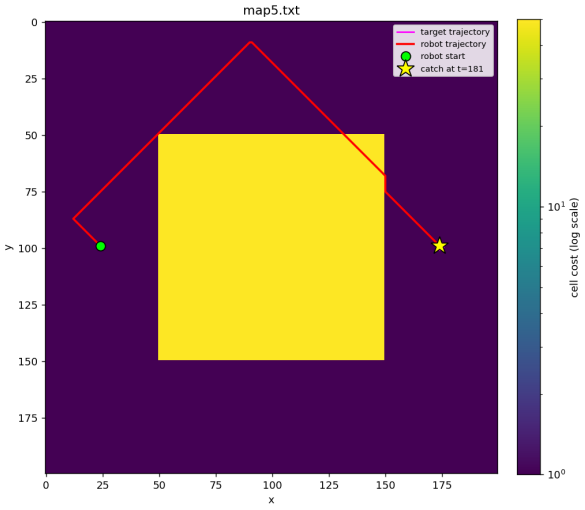


(c) Map 3

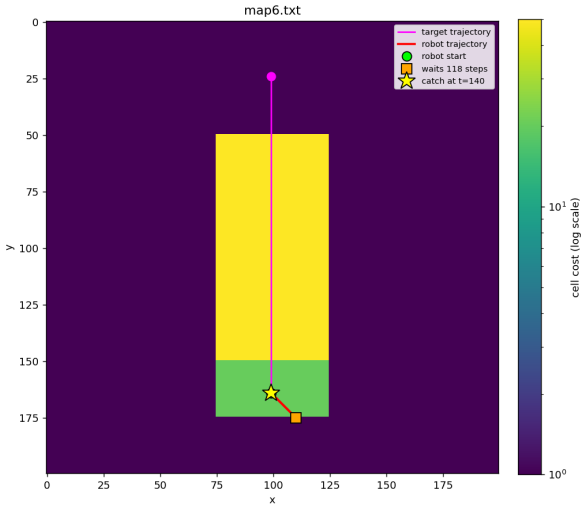


(d) Map 4

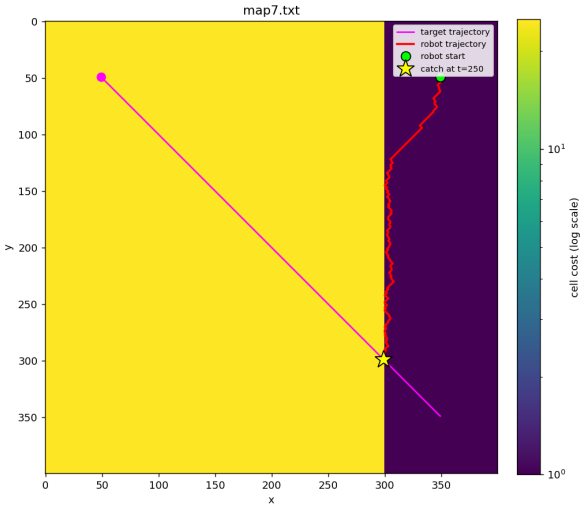
Figure 1: Robot and target trajectories, maps 1–4.



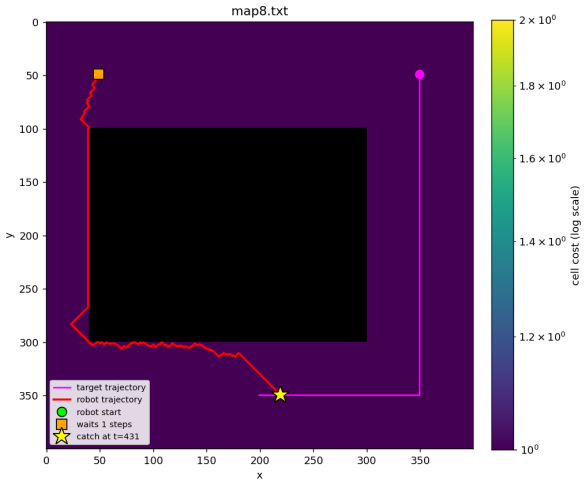
(a) Map 5



(b) Map 6

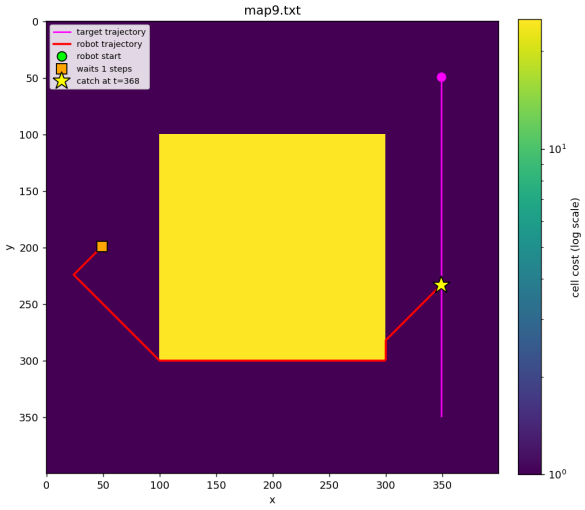


(c) Map 7



(d) Map 8

Figure 2: Robot and target trajectories, maps 5–8.



(a) Map 9

Figure 3: Robot and target trajectories, map 9.

3 Compilation and Execution Instructions

The CMakeLists.txt file was modified to add new executables to the project, so the original single-file `g++` command from the handout no longer applies. You can first build this project as given in the assignment description.

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make
```

A bash script is provided to run each map, and display a result map with the trajectory, wait times, and the point where the target was caught.

```
cd ../ # Only if you're still in the build directory, go to the main directory
chmod +x run_test.sh
run_test.sh map1 # Replace 1 with the map fo your choosing
```

The robot trajectory is written to `output/robot_trajectory.txt`. To visualize it, from the `scripts` directory:

```
python visualizer.py maps/map1.txt  # From the main directory
```

References

[1] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[2] L. R. Ford, D. R. Fulkerson. Constructing maximal dynamic flows from static flows. *Operations Research*, 6(3):419–433, 1958.

[3] S. Irnich, G. Desaulniers. Shortest path problems with resource constraints. In *Column Generation*, Springer, pp. 33–65, 2005.

[4] A. H. Land, A. G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28(3):497–520, 1960.